```
          <td class="text-right pt-2 font-bold border-t-2 border-black">
            <%= number_to_currency(cart.total_price) %>
          </td>
        </tr>
      </tfoot>
    </table>

    <%= button_to 'Empty Cart', cart, method: :delete,
      class: 'ml-4 rounded-lg py-1 px-2 text-white bg-green-600' %>
  </div>
```

That's a lot simpler. The `render` method will iterate over any collection that's passed to it. The partial template is simply another template file (by default in the same directory as the object being rendered and with the name of the table as the name). However, to keep the names of partials distinct from regular templates, Rails automatically prepends an underscore to the partial name when looking for the file. That means the partial is named `_line_item.html.erb` and can be found in the `app/views/line_items` directory:

rails80/depot_j/app/views/line_items/_line_item.html.erb

```
<tr>
  <td class="text-right"><%= line_item.quantity %></td>
  <td>&times;</td>
  <td class="pr-2">
    <%= line_item.product.title %>
  </td>
  <td class="text-right font-bold">
    <%= number_to_currency(line_item.total_price) %>
  </td>
</tr>
```

Something subtle is going on here. Inside the partial template, we refer to the current object by using the variable name that matches the name of the template. In this case, the partial is named `line_item`, so inside the partial we expect to have a variable called `line_item`.

So now we've tidied up the cart display, but that hasn't moved it into the sidebar. To do that, let's revisit our layout. Since we already have a partial